

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.10

SORT

 Copy as Markdown

Sort

Sorting puts the results of a database query in a defined order. The `find` method has an `orderBy` parameter where you can specify a column for sorting. The parameter takes a callback as an argument. The callback receives a model-specific table descriptor, conventionally named `t`, though you can name the parameter anything. The same descriptor is also available as the static `t` field on the model, and that name is fixed. The table descriptor represents the model's database table and includes a field for each column. In the callback, return the column to sort by:

```
var companies = await Company.db.find(  
  session,  
  orderBy: (t) => t.name,  
);
```

In the example we fetch all companies and sort them by their name.

By default, results are sorted in ascending order, so a bare column is equivalent to calling `.asc()` on it. To sort in descending order, call `.desc()` on the column:

```
var companies = await Company.db.find(  
  session,  
  orderBy: (t) => t.name.desc(),  
);
```

To order by several different columns, use `orderByList` instead. It cannot be used in conjunction with `orderBy`.

```
var companies = await Company.db.find(  
  session,  
  orderByList: (t) => [t.name.desc(), t.id.asc()],  
);
```



Ask AI

In the example we sort companies by their name in descending order, and then by their id in ascending order.

The `orderBy` and `orderByList` parameters are not limited to `find`. They also appear on `updateWhere`, `delete`, and `deleteWhere`, where they order the returned rows. When [paginating](#) with `limit` and `offset`, always set an `orderBy` so pages stay consistent between requests.

Sort on relations

To sort based on a field from a related model, use the chained field reference.

```
var companies = await Company.db.find(
  session,
  orderBy: (t) => t.ceo.name,
);
```

In the example we fetch all companies and sort them by their CEO's name.

You can order results based on the count of a list relation (1:n).

```
var companies = await Company.db.find(
  session,
  orderBy: (t) => t.employees.count(),
);
```

In the example we fetch all companies and sort them by the number of employees.

The count used for sorting can also be filtered using a sub-filter.

```
var companies = await Company.db.find(
  session,
  orderBy: (t) => t.employees.count(
    (employee) => employee.role.equals('developer'),
  ),
);
```

In the example we fetch all companies and sort them by the number of employees with the role of "developer".

Vector and geography fields can be ordered by distance to a query value. See [Vector distance operators](#) and [Geography operators](#).